國立臺北科技大學
National Taipei University of Technology

資料結構
Data Structure

Lab 09

姓名:王顗程
學號: 114310215

Lab09 (題目編號)
Exercise: Delete Node in a Binary Search Tree
Code

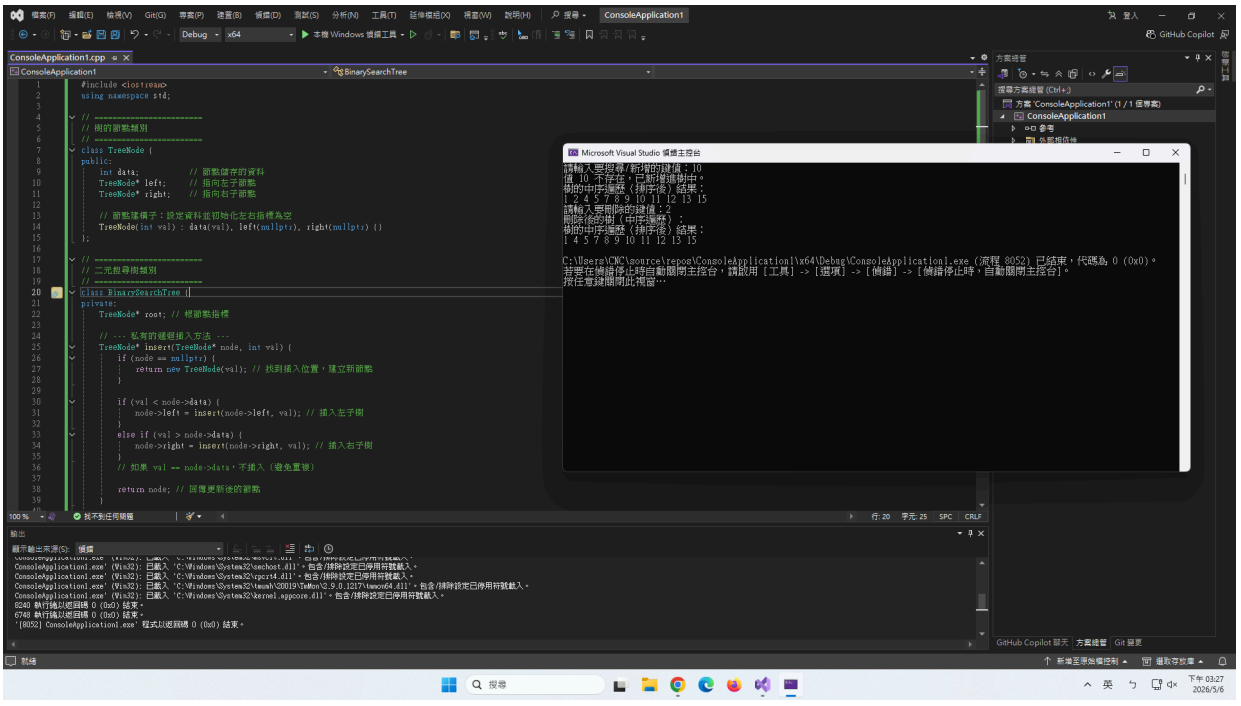
```
#include <iostream>
using namespace std;

// ========================
// 樹的節點類別
// ========================
class TreeNode {
public:
   int data;        // 節點儲存的資料
   TreeNode* left;    // 指向左子節點
   TreeNode* right;   // 指向右子節點

   // 節點建構子：設定資料並初始化左右指標為空
   TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// ========================
// 二元搜尋樹類別
// ========================
class BinarySearchTree {
private:
   TreeNode* root; // 根節點指標

   // --- 私有的遞迴插入方法 ---
   TreeNode* insert(TreeNode* node, int val) {
      if (node == nullptr) {
         return new TreeNode(val); // 找到插入位置，建立新節點
      }

      if (val < node->data) {
         node->left = insert(node->left, val); // 插入左子樹
      }
      else if (val > node->data) {
         node->right = insert(node->right, val); // 插入右子樹
      }
      // 如果 val == node->data，不插入（避免重複）

      return node; // 回傳更新後的節點
   }
```

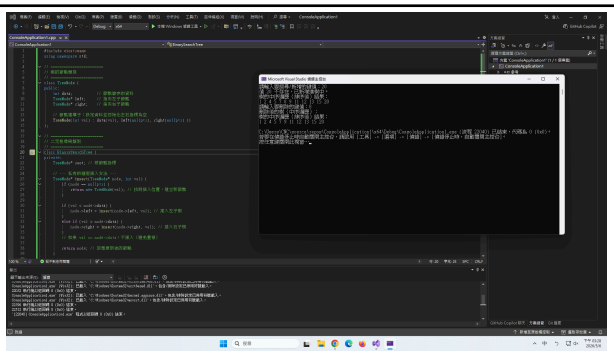
```
// --- 私有的遞迴搜尋方法 ---
bool search(TreeNode* node, int target) {
    if (node == nullptr) return false;

    if (node->data == target) return true;
    else if (target < node->data) return search(node->left, target); // 繼續在左子樹找
    else return search(node->right, target);// 繼續在右子樹找
}

// --- 私有的中序遍歷方法(左-根-右)---
void inOrderTraversal(TreeNode* node) {
    if (node == nullptr) return;

    inOrderTraversal(node->left);  // 先走訪左子樹
    cout << node->data << " ";  // 印出目前節點的資料
    inOrderTraversal(node->right); // 再走訪右子樹
}

// --- 私有的刪除方法 ---
TreeNode* deleteNode(TreeNode* node, int val) {
    if (node == nullptr) return nullptr;

    // 1. 尋找要刪除的節點
    if (val < node->data) {
        node->left = deleteNode(node->left, val);
    }
    else if (val > node->data) {
        node->right = deleteNode(node->right, val);
    }
    // 找到目標節點
    else {
        // 情況 1 & 2: 只有一個子節點或無子節點
        if (node->left == nullptr) {
            TreeNode* temp = node->right;
            delete node;
            return temp;
        }
        else if (node->right == nullptr) {
            TreeNode* temp = node->left;
            delete node;
            return temp;
        }
        // 情況 3: 有兩個子節點
```

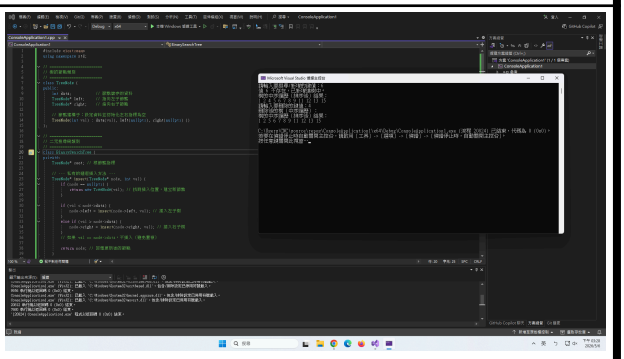
```
        // 規則：使用右子樹的最小值（In-order Successor）來替換
        TreeNode* temp = findMin(node->right);

        // 將數值複製過來
        node->data = temp->data;

        // 刪除右子樹中那個被拿來遞補的節點
        node->right = deleteNode(node->right, temp->data);
    }
    return node;
}
// --- 私有的找最小值方法 ---
TreeNode* findMin(TreeNode* node) {
    while (node->left != nullptr) {
        node = node->left;
    }
    return node;
}

public:
// --- 建構子：初始化為空樹 ---
BinarySearchTree() : root(nullptr) {}

// --- 公開插入方法 ---
void insert(int val) {
    root = insert(root, val);
}

// --- 公開搜尋方法 ---
bool search(int target) {
    return search(root, target);
}

// --- 公開中序遍歷方法 ---
void displayInOrder() {
    cout << "樹的中序遍歷（排序後）結果：" << endl;
    inOrderTraversal(root);
    cout << endl;
}

// --- 公開刪除方法 ---
void deleteValue(int val) {
    root = deleteNode(root, val);
```

```
    }
};

int main() {
    BinarySearchTree bst; // 建立一棵空的樹

    // 預設插入的資料
    int values[] = { 7, 1, 4, 2, 8, 13, 12, 11, 15, 9, 5 };
    int n = sizeof(values) / sizeof(values[0]);

    // 把資料插入樹中
    for (int i = 0; i < n; ++i) {
        bst.insert(values[i]);
    }
    // 讓使用者輸入要插入的鍵值
    int key;
    cout << "請輸入要搜尋/新增的鍵值:";
    cin >> key;
    cin.clear();
    cin.ignore(1000, '\n');
    if (bst.search(key)) {
        cout << "值 " << key << " 已存在樹中，不新增。" << endl;
    }
    else {
        cout << "值 " << key << " 不存在，已新增進樹中。" << endl;
        bst.insert(key);
    }

    // 顯示目前樹的中序遍歷結果
    bst.displayInOrder();

    // 讓使用者輸入要刪除的鍵值
    int deleteKey;
    cout << "請輸入要刪除的鍵值:";
    cin >> deleteKey;

    // 刪除節點
    bst.deleteValue(deleteKey);

    // 顯示刪除後的樹
    cout << "刪除後的樹(中序遍歷):" << endl;
    bst.displayInOrder();
```

```
    return 0;
}
```


ConsoleApplication1.cpp
#include <iostream>
using namespace std;
// ==========
// 樹的節點類別
// ==========
class TreeNode {
public:
int data; // 節點儲存的資料
TreeNode* left; // 指向左子節點
TreeNode* right; // 指向右子節點
// 節點建構子：設定資料並初始化左右指標為空
TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};
// ==========
// 二元搜尋樹類別
// ==========
class BinarySearchTree {
private:
TreeNode* root; // 根節點指標
// --- 私有的遞迴插入方法 ---
TreeNode* insert(TreeNode* node, int val) {
if (node == nullptr) {
return new TreeNode(val); // 找到插入位置，建立新節點
}
if (val < node->data) {
node->left = insert(node->left, val); // 插入左子樹
}
else if (val > node->data) {
node->right = insert(node->right, val); // 插入右子樹
}
// 如果 val == node->data，不插入（避免重複）
return node; // 回傳更新後的節點
}
Microsoft Visual Studio 偵錯主控台
請輸入要搜尋/新增的鍵值：10
值 10 不存在，已新增進樹中。
樹的中序遍歷（排序後）結果：
1 2 4 5 7 8 9 10 11 12 13 15
請輸入要刪除的鍵值：2
刪除後的樹（中序遍歷）：
樹的中序遍歷（排序後）結果：
1 4 5 7 8 9 10 11 12 13 15
C:\Users\CNC\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (流程 8052) 已結束，代碼為 0 (0x0)。
若要在偵錯停止時自動關閉主控台，請啟用 [工具] -> [選項] -> [偵錯] -> [偵錯停止時，自動關閉主控台]。
按任意鍵關閉此視窗…





Lab01-Q1. (題目編號)
Code
Result
Discussion

Lab01-Q#. (題目編號)
Briefly describe the problem. (題目簡述)
Code

Paste your code here. Please use Pygments to highlight your code. (請使用Pygments傳換程式碼格式後再貼上)	Paste your code here. Please use Pygments to highlight your code. (請使用Pygments傳換程式碼格式後再貼上)

result		
(a)	(b)	(c)
	Show your result in tables or screenshots. (請使用表格或是截圖呈現結果。)	

Discussion
Discuss and conclude your results, or answer questions. (實驗討論、結論或回答問題。)